

UNIVERSIDADE
LUSÓFONA

Relatorio Sistemas Operativos

- Análise Comparativa de Algoritmos de Escalonamento de Processos -

Relatório de Laboratório

Sistemas Operativos

Joao Victor Franco da Costa, a22601053, LEI

Professor de UC: Martijn Kuipers
Professor de UC: Paulo Guedes
Professor de UC: Daniel Silveira

Departamento de Engenharia Informática e Sistemas de Informação
Universidade Lusófona, Centro Universitário de Lisboa

Outubro 2026

www.ulusofona.pt

Direitos de cópia

Relatorio Sistemas Operativos, Copyright © de Joao Victor Franco da Costa, Universidade Lusófona

A Escola de Comunicação, Arquitectura, Artes e Tecnologias da Informação (ECATI) e a Universidade Lusófona (UL) têm o direito, perpétuo e sem limites geográficos, de arquivar e publicar este relatório através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, e de a divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objectivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

Índice

Índice iii

1 Introdução e Contextualização Tecnológica 2

2 Panorama dos Algoritmos de Escalonamento 3
2.1 First-In, First-Out (FIFO) 3
2.2 Shortest Job First (SJF) 3
2.3 SHORTEST REMAINING TIME FIRST (SRTF) 3
2.4 Round-Robin (RR) 3
2.5 Multi-Level Feedback Queue (MLFQ) 4

3 Tabela de Desempenho teórico 5

4 Exemplos teóricos 6
4.1 Metodologia e Configuração dos Testes Comparativos 6
4.2 Primeiro teste 6
4.3 Segundo Teste 8
4.4 Teste Final 12

5 Conclusão 14

1. Introdução e Contextualização Tecnológica

O escalonador de CPU é um dos componentes mais críticos de um Sistema Operativo. A sua função principal é determinar a ordem de execução das tarefas na fila de prontos, procurando otimizar métricas fundamentais como o **Turnaround Time** (tempo total no sistema), o **Response Time** (latência até ao primeiro processamento), o **Waiting Time** (tempo em fila) e a utilização global do processador.
Neste trabalho, analisamos quatro estratégias de escalonamento, evoluindo do conceito mais simples até ao mais dinâmico e complexo que tivemos conhecimento até o presente momento: **First-In, First-Out** (FIFO), **Shortest Job First** (SJF), **Round-Robin** (RR) e **Multi-Level Feedback Queue** (MLFQ).

É relevante notar que a implementação do MLFQ utilizada nas simulações deste trabalho possui aproximações e limitações em relação aos algoritmos MLFQ de nível de produção:

1. **Estrutura de Dados Única:** Em vez de gerir 5 filas de prioridade fisicamente independentes, a nossa simulação utiliza uma única fila encadeada/ligada. O escalonador percorre ciclicamente a lista à procura do PCB com a prioridade mais alta disponível.
2. **Tratamento Incompleto de Tarefas I/O Intensive:** A nossa lógica atual não considera o comportamento de processos fortemente voltados a I/O (**I/O-bound**) que libertam o CPU voluntariamente antes de esgotar o *quantum*. Em sistemas reais, esses processos mantêm-se em filas de alta prioridade para favorecer a interatividade. Na simulação atual, a lógica de rebaixamento foca-se essencialmente na exaustão acumulada do tempo de execução das tarefas.

2. Panorama dos Algoritmos de Escalonamento

2.1 First-In, First-Out (FIFO)

O FIFO é o algoritmo de escalonamento mais elementar, operando sob uma política não-preemptiva estrita.

As tarefas são processadas exatamente pela ordem de chegada na fila de prontos. Apresenta uma implementação extremamente simples, sem nenhuma lógica de decisão para a escolha dos processos e zero trocas de contexto desnecessárias. Funciona bem quando todas as tarefas possuem tempos de CPU semelhantes.

Se um processo longo chegar em primeiro lugar, todas as tarefas curtas subsequentes ficam bloqueadas atrás dele. Isto faz disparar o tempo médio de espera e o tempo de resposta do sistema. Ou seja, tarefas que possuem uma quantidade de ENTRADA/SAÍDA muito intensa serão tragicamente penalizadas de acordo com os processos que estão à sua frente.

Em relação à latência, este presenta a maior latência média e imprevisível para tarefas que chegam após processos longos.

2.2 Shortest Job First (SJF)

O SJF aborda a principal limitação do FIFO ao priorizar tarefas com base no seu tempo total de processamento necessário.

Ele seleciona sempre a tarefa com a menor duração estimada de execução e é ótimo para minimizar o tempo médio de espera das tarefas e o de execução. Sua falha pode ser observada em processos longos, que podem vir a sofrer starvation (nunca executam) se houver um fluxo contínuo de tarefas curtas a entrar na fila. (porém em nosso projeto isso é muito difícil de ser observado, já que não é possível simularmos com precisão tarefas altamente interativas).

Em relação à metrica das latências a melhora é significativa se compararmos com as de tarefas curtas no modelo FIFO, mas penaliza fortemente tarefas longas.

2.3 SHORTEST REMAINING TIME FIRST (SRTF)

NOTA: Dado o seu parentesco direto com o SJF, optei por incluir a implementação e análise do SRTF como complemento ao estudo, ainda que o mesmo não integrasse os requisitos obrigatórios do trabalho.

Este algoritmo se propõe a solucionar um problema que temos ao executarmos o SJF, que é, ao termos uma tarefa I/O, apesar de seu tempo de execução ser baixo, em um cenário onde já temos uma tarefa CPU BOUND em processamento pelo CPU, esta tarefa estará refém do termino de execução do processo anterior. Culminando assim em um ganho de desempenho muito marginal se comparado com a FIFO num cenário real em que não temos controle da ordem de entrada e saída das tarefas.

Por outro lado, abrimos espaço para um comportamento "malicioso", que é tarefas altamente interativas ocasionalmente acarretarão num problema de STARVATION para tarefas CPU BOUND que estarão sempre em espera, dando vez à tarefas curtas serem executadas.

Em resumo: com relação a latência, é o algoritmo que de longe teria as melhores métricas SE todas as tarefas que houvessem fossem I/O bound, o que não é realidade. Este problema então só é solucionado, "medianamente", através de Round-Robin ou, de forma mais completa, pela implementação de uma MLFQ ou CFS.

2.4 Round-Robin (RR)

O round-robin introduz a preempção temporizada, visando a interatividade e a equidade (fairness), mas que ainda acaba por não ser suficiente pois não leva em consideração a "urgência" com que algumas tarefas necessitam ser processadas

Cada processo recebe uma fatia fixa de tempo no CPU chamados *quantum*. Se a tarefa não terminar no seu *quantum*, sofre preempção e é colocada no fim da fila de prontos. Desta forma, todas as tarefas terão uma fatia semelhante de tempo que podem usufruir do processador.

É bom para sistemas interativos e garante que nenhum processo monopolize o CPU e elimina a inanição, porém, falha ao tratar processos CPU BOUND e I/O da mesma maneira que o outro. Se o quantum for demasiado pequeno, o excesso de trocas de contexto gera um elevado overhead, degradando a eficiência do CPU. Se o quantum for muito grande, o comportamento degenera para o FIFO.

Apresenta a menor latência do grupo básico, pois garante que todo o processo obtém CPU num intervalo previsível.

2.5 Multi-Level Feedback Queue (MLFQ)

Dentre os 3 vistos anteriormente, o MLFQ é uma abordagem avançada que leva em consideração o comportamento dos processos em tempo real, tentando combinar os benefícios do SJF (otimização de tarefas curtas) e do RR (equidade e interatividade).

Mantém múltiplas filas prioritárias independentes (que no nosso simulador é apenas uma fila, onde o próximo elemento a ser escolhido é sempre o que possui maior prioridade nesta fila).

Processos novos entram sempre com o maior número de prioridade. Se consomem todo o *quantum* disponível durante este nível de prioridade que se encontra, recebe uma demoção, abrindo espaço para as próximas tarefas que enventualmente possam entrar na READYQUEUE.

A ideia é que ela se adapte dinamicamente a cargas de trabalho mistas sem necessitar de conhecimento prévio da duração das tarefas já que todas elas receberão maior prioridade possível e um *quantum* limitado para execução.

Exige uma afinação complexa de parâmetros como *quantum* por fila, políticas de demoção/promoção, mecanismos contra starvation como o priority boost e também mecanismos que impeçam tarefas de trapacearem e se manterem para sempre na fila de maior prioridade por serem I/O intensive (que não foi levada em consideração neste projeto).

Da forma que fomos instruídos a implementa-la, ainda há um comportamento falho que não leva em consideração tarefas que não consomem todo o seu *quantum*. Penalizando, assim, tarefas que são I/O Intensive. Desta forma, temos quase um RR levemente mais inteligente mas que ainda assim carece de alguns refinamentos.

3. Tabela de Desempenho teórico

Para esta avaliação, considera-se abaixo o comportamento de cada algoritmo sob a perspectiva ideal de seu funcionamento. Destaca-se o MLFQ em sua especificação teórica completa: um mecanismo que não apenas prioriza tarefas interativas, mas considera também o *quantum accounting* (acumulação do tempo de CPU consumido no mesmo nível de prioridade). Isso impede que processos **I/O-bound** mal-intencionados "trapaceiem" ao realizar um pequeno I/O antes do fim do **quantum** para permanecerem indefinidamente na fila de topo.

Vale a pena também destacar que o modelo de scheduler usado para a SJF no trabalho é o não preemptivo, tornando-o quase uma FIFO que prioriza processos curtos. Uma vez que um processo longo já está sendo processado pelo CPU, a alocação de uma nova tarefa na READY QUEUE não gera a devida preempção do processo já em execução, beneficiando massivamente processos grandes que por sua vez já foram agarrados pelo CPU.

Algoritmo	**Response T. (Latência)**	**Waiting Time**	**Turnaround Time**	**Perfil Favorável**	**Trocas de Contexto**	**Comportamento e Mapeamento de Riscos**
FIFO	Pessimista / Elevado	Elevado	Altamente dependente do CPU time total	CPU-bound (Lote / Batch)	Mínimo	Sujeito ao **Convoy Effect**. Prejudica gravemente **I/O-bound** se houver processos **CPU-bound** longos à frente.
SJF	Moderado	Mínimo (Ótimo)	Depende da tarefa que já está sendo processada anteriormente	Tarefas Curtas / Variadas	Mínimo	Minimiza o **waiting time** global, mas causa **starvation** em tarefas longas **CPU-bound** na presença de fluxo contínuo de tarefas curtas.
RR	Baixo / Rápido	Moderado	Elevado para tarefas longas	I/O-bound / Interativo	Muito Elevado	Garante equidade (**fairness**). Se o **quantum** for muito pequeno, o **overhead** de troca de contexto degrada o **throughput** do sistema.
MLFQ (Ideal)	Mínimo / Rápido	Baixo / Otimizado	Próximo do ótimo (SJF)	Misto (I/O e CPU-bound)	Elevado porém dinâmico	Diferencia **I/O-bound** de **CPU-bound**. O **accounting** impede trapaças de I/O, despromovendo processos por tempo total acumulado em CPU.

4. Exemplos teóricos

4.1 Metodologia e Configuração dos Testes Comparativos

Para a bateria inicial de testes comparativos, foram selecionadas duas tarefas com o mesmo tempo total de processamento acumulado (24000ms de CPU Time), mas com padrões de execução distintos:

Tarefa 1: (CPU-Bound): Composta por 3 bursts longos de 8000 ms cada, sem tempos de bloqueio/espera, simulando um processo intensivo de cálculo.

Tarefa 2 (I/O-Bound / Interativa): Composta por 24 bursts curtos de 1000ms, intercalados por um tempo de bloqueio de 10ms entre requisições, simulando um comportamento interativo com entradas e saídas frequentes da CPU.

4.2 Primeiro teste

O objetivo desta abordagem é isolar o volume de trabalho e avaliar estritamente como a política de cada escalonador lida com a alternância de prioridades, tempos de resposta e overhead de trocas de contexto. Posteriormente, os cenários serão extrapolados para cargas concorrentes mais complexas a fim de aprofundar os resultados obtidos.

Para a realização dos testes, assume-se um quantum de 200ms para os algoritmos Round Robin e MLFQ. Adicionalmente, no caso da MLFQ, assume-se um intervalo de 3000ms para a aplicação do priority boost, garantindo a prevenção de starvation e permitindo observar claramente a dinâmica de demoção e promoção de prioridades.

4.2.1 FIFO

Relatório de Desempenho — FIFO

PID	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
2757	3286 ms	3297 ms	29869 ms	24000 ms	26583 ms	11 ms	2583 ms	3
2758	3286 ms	11437 ms	52594 ms	24000 ms	49308 ms	8151 ms	25308 ms	24

Turnaround Médio: 37945.50 ms
Tempo Resp. Médio: 4081.00 ms
Tempo Espera Médio: 13945.50 ms
Eficiência Média: 63.25%
Trocas de Contexto/Tarefa: 13.50
Utilização da CPU: 86.26%

No algoritmo FIFO, podemos observar o impacto do convoy effect. A primeira tarefa (pid2757) ao alocar o processador mantém a CPU por 8000ms contínuos, forçando a tarefa interativa a aguardar na Ready Queue a cada requisição de I/O. Isso resulta num tempo de resposta médio extremamente elevado (4081.00ms), demonstrando a ineficiência do algoritmo em ambientes com fluxos mistos de CPU e I/O, apesar de manter poucas trocas de contexto (13.50)isto é, apesar de deixar as tarefas executarem o máximo que quiserem.

4.2.2 Shortest Job First

Relatório de Desempenho — SJF (Não-Preemptivo)

PID	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
2779	2097 ms	3124 ms	29680 ms	24000 ms	27583 ms	1027 ms	3583 ms	3
2780	2097 ms	2107 ms	51370 ms	24000 ms	49273 ms	10 ms	25273 ms	24

Turnaround Médio: 38428.00 ms
Tempo Resp. Médio: 518.50 ms
Tempo Espera Médio: 14428.00 ms
Eficiência Média: 62.45%
Trocas de Contexto/Tarefa: 13.50
Utilização da CPU: 88.20%

No cenário avaliado, o SJF não-preemptivo apresentou métricas globais praticamente idênticas ao FIFO (38428.00ms de Turnaround médio contra 37945.50ms). Como ambas as tarefas possuem o mesmo tempo total de processamento de 24000ms e o algoritmo não realiza preempção, assim que a tarefa CPU-bound ganha o processador, ela executa o seu burst longo ininterruptamente. Desta forma, a tarefa interativa acaba por ficar retida da mesma maneira, tornando o SJF incapaz de oferecer um ganho significativo em relação ao FIFO neste exemplo.

Um bom cenário para estas tarefas curtas seria onde tivéssemos uma entrada massiva de múltiplos processos pequenos diferentes e ocasionalmente um longo. Desta forma, caso os curtos já encontrassem-se na READY-QUEUE, o CPU-Bound seria submetido a um grande tempo de espera.

4.2.3 Round Robin

Relatório de Desempenho — Round Robin (RR - Quantum: 200ms)

PID	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
2801	3712 ms	3722 ms	51858 ms	24000 ms	48146 ms	10 ms	24146 ms	117
2802	3712 ms	3936 ms	52572 ms	24000 ms	48860 ms	224 ms	24860 ms	117

Turnaround Médio: 48503.00 ms
Tempo Resp. Médio: 117.00 ms
Tempo Espera Médio: 24503.00 ms
Eficiência Média: 49.48%
Trocas de Contexto/Tarefa: 117.00
Utilização da CPU: 86.30%

Com a introdução da preemptação por quantum de 200ms, o Round Robin impôs um regime de equidade no compartilhamento de tempo de CPU. O tempo de resposta médio caiu drasticamente para 117.00ms, favorecendo bastante o tempo de reação inicial da aplicação interativa. Contudo, essa alternância frequente introduziu um overhead massivo no sistema, elevando a média de trocas de contexto para 117.00 por tarefa e degradando o Turnaround médio em relação aos algoritmos não-preemptivos. Tal métrica só poderia ser abordada eficientemente se usássemos um sistema em que tarefas interativas tivessem mais prioridade para serem executadas por curto períodos ao passo em que tarefas longas tivessem mais tempo para se manterem em execução. (Que é o caso de uma MLFQ completamente implementada)

4.2.4 Multi-Level Feedback Queue

Relatório de Desempenho — MLFQ (Multi-Level Feedback Queue)

PID	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
2824	2149 ms	2363 ms	43709 ms	24000 ms	41560 ms	214 ms	17560 ms	120
2823	2149 ms	2160 ms	51072 ms	24000 ms	49238 ms	11 ms	24923 ms	119

Turnaround Médio: 45241.50 ms
Tempo Resp. Médio: 112.50 ms
Tempo Espera Médio: 21241.50 ms
Eficiência Média: 53.05%
Trocas de Contexto/Tarefa: 119.50
Utilização da CPU: 88.70%

Já a nossa MLFQ apresentou resultados bastante similares ao Round Robin (112.50ms de tempo de resposta e 119.50 trocas de contexto) devido a limitações no desenho da simulação atual. Como a implementação utiliza um quantum uniforme em todas as filas (sem compensação de escala para filas inferiores) e ambas as tarefas deram entrada simultâneamente no simulador, as duas acabaram por ser despromovidas sincronazamente para a prioridade mais baixa de forma irrealista. Para evidenciar o real benefício da MLFQ, que preserva a alta prioridade para tarefas interativas curtas e penaliza o consumo contínuo, o próximo teste reformulará a entrada de dados, substituindo a tarefa interativa de 24 bursts por 24 aplicações distintas de 1000ms simulando reentradas periódicas na prioridade máxima.

4.3 Segundo Teste

Para a segunda bateria de testes, a fim de tentar preservar a clareza visual das tabelas sem abrir mão de uma amostragem coerente das especificidades de cada escalonador, optei por reportar apenas 3 tarefas: a tarefa CPU-bound, a tarefa de I/O de melhor desempenho e a de pior desempenho. A classificação destas duas últimas foi definida com base no menor e no maior tempo de resposta (latência) registado pela tarefa durante a sua execução.

4.3.1 FIFO

Relatório de Desempenho — FIFO

PID / Categoria	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
3002 (CPU--Bound)	2914 ms	2925 ms	51872 ms	24000 ms	48958 ms	11 ms	24958 ms	3
3004 (Melhor IO)	4914 ms	11062 ms	12081 ms	1000 ms	7167 ms	6148 ms	6167 ms	1
3050 (Pior IO)	16481 ms	42619 ms	43638 ms	1000 ms	27157 ms	26138 ms	26157 ms	1

Turnaround Médio: 18103.08 ms
Tempo Resp. Médio: 15167.84 ms
Tempo Espera Médio: 16183.08 ms
Eficiência Média: 10.61%
Trocas de Contexto/Tarefa: 1.08
Utilização da CPU: 87.40%

Nessa nova abordagem, a submissão separada das 24 tarefas curtas não alterou a limitação da FIFO que já havíamos relatado antes. A tarefa CPU-bound chegou primeiro e reteve o processador continuamente durante cada um dos seus bursts longos de 8000ms. As novas tarefas interativas, ao darem entrada no

sistema enquanto um burst longo estava em curso, acumularam-se sequencialmente na fila de espera. O tempo de resposta médio manteve-se extremamente alto (15167.84ms).

4.3.2 Shortest Job First

Com o envio assíncrono das tarefas, o SJF (não-preemptivo) apresentou uma pequena melhoria no tempo de turnaround médio de 14543.08m se em comparação com 18103.08ms da FIFO, mas ainda assim insuficiente para caracterizar uma grande melhoria de desempenho. Nesse caso, como o SJF não é preemptivo, quando uma tarefa curta chega, ela não consegue interromper o burst de 8000ms da tarefa CPU-bound em execução, então não há muito o que se esperar. O ganho ocorre apenas nas pequenas janelas em que o processador fica livre das tarefas longas enquanto já existem múltiplas tarefas curtas acumuladas na fila, permitindo assim ordenar as de menor tamanho antes, embora a latência geral continue comprometida.

Relatório de Desempenho — SJF

PID / Categoria	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
3072 (CPU--Bound)	2627 ms	2637 ms	51719 ms	24000 ms	49092 ms	10 ms	25092 ms	3
3074 (Melhor IO)	4630 ms	10777 ms	11796 ms	1000 ms	7166 ms	6147 ms	6166 ms	1
3120 (Pior IO)	16188 ms	34204 ms	35223 ms	1000 ms	19035 ms	18016 ms	18035 ms	1

Turnaround Médio: 14543.08 ms
Tempo Resp. Médio: 11601.96 ms
Tempo Espera Médio: 12623.08 ms
Eficiência Média: 13.20%
Trocas de Contexto/Tarefa: 1.08
Utilização da CPU: 87.63%

Apenas para fins comparativos, neste mesmo teste, com tarefas idênticas, com o modelo SRTF, o tempo de latência médio despencou para 183.36ms, o turnaround médio para 3126.48ms, o waiting time para 1206.48. O problema é que os dados acabam sendo extremamente enviezados pela quantidade de tarefas pequenas que conseguiram concluir rapidamente seus bursts, enquanto isso a tarefa de longa duração só teve chance de ser executada por conta do termino das entradas interativas. Num cenário real, em que não é possível prever nem a entrada nem a quantidade de tarefas interativas que ocorrerão no futuro, este scheduler NUNCA iria executar tarefas CPU-BOUND.

4.3.3 Round Robin

No Round Robin, a divisão do tempo por quantum de 200ms manteve a distribuição equilibrada de CPU entre a tarefa CPU-bound e o conjunto das tarefas interativas (que juntas somam aproximadamente 50% do volume de processamento total do cenário).

Relatório de Desempenho — Round Robin

PID / Categoria	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
3142 (CPU-Bound)	7055 ms	7066 ms	56195 ms	24000 ms	49140 ms	11 ms	25140 ms	16
3144 (Melhor IO)	9052 ms	9205 ms	12573 ms	1000 ms	3521 ms	153 ms	2521 ms	5
3190 (Pior IO)	20626 ms	24720 ms	36184 ms	1000 ms	15558 ms	4094 ms	14558 ms	5

Turnaround Médio: 15929.96 ms
Tempo Resp. Médio: 2135.64 ms
Tempo Espera Médio: 14009.96 ms
Eficiência Média: 12.05%
Trocas de Contexto/Tarefa: 5.44
Utilização da CPU: 81.02%

Não há muito o que ser elaborado, o Round Robin sempre distribuirá o CPU de forma igualitária. No entanto, certas métricas agregadas mascaram algumas ineficiências e nos impedem de declarar este escalonador ou "aquele" como universalmente superior. Algumas destas métricas são difíceis de serem avaliadas por si só. Dividir o tempo de processamento de maneira engessadamente uniforme para toda e qualquer tarefa não garante uma equidade efetiva no sistema, e nem eficiência. Num cenário real, existem tarefas com diversos perfis distintos: algumas libertam a CPU muito antes de esgotar o quantum, enquanto outras exigem resposta imediata para manter a fluidez da aplicação. E esta reatividade instantânea não é alcançável pelo Round Robin, pois uma tarefa interativa recém-chegada é forçada a aguardar a sua vez na fila atrás de processos que consomem o quantum na íntegra e as vezes nem era tão urgente como outra.

4.3.4 Multi-Level Feedback Queue

Já no cenário da MLFQ, mesmo considerando as limitações desse ambiente simulado em relação a uma implementação completa de um SO comercial, já é possível notar as vantagens desta abordagem. O tempo de resposta registado aproxima-se de algoritmos focados em tarefas curtas (como o SRTF), enquanto as tarefas CPU-bound não sofrem de postergação infinita.

Relatório de Desempenho — MLFQ

PID / Categoria	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
3282 (CPU--Bound)	2640 ms	2651 ms	49186 ms	24000 ms	46546 ms	11 ms	22546 ms	116
3286 (Melhor IO)	24641 ms	24661 ms	28479 ms	1000 ms	3838 ms	20 ms	2838 ms	5
3328 (Pior IO)	45695 ms	46712 ms	51526 ms	1000 ms	5831 ms	1017 ms	4831 ms	5

Turnaround Médio: 7667.80 ms
Tempo Resp. Médio: 240.56 ms
Tempo Espera Médio: 5747.80 ms
Eficiência Média: 25.04%
Trocas de Contexto/Tarefa: 9.44
Utilização da CPU: 87.62%

Como demonstrado na amostragem, o pior caso entre as tarefas interativas (Pior I/O) concluiu a sua execução até mesmo 2 segundos após a conclusão do processo longo (PID 3282). O tempo de Turnaround médio (7667.80ms) foi o menor entre todos os escalonadores analisados, mesmo num cenário onde as tarefas curtas possuem 1000ms de tempo de CPU, que é um valor substancialmente elevado quando se comparado com interações reais de I/O. Em resumo, o que é crítico e urgente é atendido de forma imediata sem que o trabalho de tarefas de longa duração seja excessivamente penalizado.

Numa implementação completa de MLFQ, os processos longos seriam ainda mais otimizados por meio de quantums progressivamente maiores nas filas de menor prioridade, reduzindo o overhead de trocas de contexto. Além disso, mecanismos como o Priority Boost periódico promoveriam tarefas retidas nas filas inferiores para prevenir a starvation, ao passo que limites estritos de consumo de tempo e reentrada impediriam que tarefas interativas persistentes monopolizassem a fila de maior prioridade, garantindo uma ainda maior equidade global do sistema.

4.4 Teste Final

Para este último teste, com intuito de demonstrar melhor a MLFQ será utlizado:
.Uma tarefa de um burst único e longo de 20000ms introduzida primeiro no sistema;
.Uma tarefa com 3 bursts de 6660ms introduzidas também no início;
.Dez tarefas com bursts de 1000ms introduzidas aleatoriamente;
.Dez tarefas com bursts de 200ms introduzidas aleatoriamente;
.Des tarefas com 3 bursts de 75ms introduzidas aleatoriamente;

Relatório Geral do Teste — MLFQ (Cenário Aleatórizado)

PID / Perfil	Chegada	1ª Exec	Fim	CPU Time	Turnaround	Resp. T.	Wait T.	Context
1. Tarefas Longas de Fundo (CPU-Bound / Baixa Prioridade)								
3748 (Fundo A)	3338 ms	3348 ms	58684 ms	20000 ms	55346 ms	10 ms	35346 ms	93
3749 (Fundo B)	3338 ms	3552 ms	58633 ms	19980 ms	55295 ms	214 ms	35315 ms	95
2. Tarefa de Duração Média (Multi-burst)								
3756 (Média I/O)	4855 ms	4977 ms	8000 ms	1000 ms	3145 ms	122 ms	2145 ms	5
3762 (Média I/O)	7664 ms	8000 ms	11766 ms	1000 ms	4102 ms	336 ms	3102 ms	5
3. Tarefas Interativas com E/S (3x 75ms CPU + 300ms I/O)								
3768 (Interativa)	9881 ms	10167 ms	11532 ms	240 ms	1651 ms	286 ms	1411 ms	3
3774 (Interativa)	13089 ms	13191 ms	14852 ms	240 ms	1763 ms	102 ms	1523 ms	3
3819 (Interativa)	35868 ms	36234 ms	37596 ms	240 ms	1728 ms	366 ms	1488 ms	3
3828 (Interativa)	40085 ms	40238 ms	41603 ms	240 ms	1518 ms	153 ms	1278 ms	3
4. Tarefas Curtas de Pronta Resposta (200ms CPU)								
3753 (Curta CPU)	3847 ms	3959 ms	4163 ms	200 ms	316 ms	112 ms	116 ms	1
3765 (Curta CPU)	8773 ms	8814 ms	9017 ms	200 ms	244 ms	41 ms	44 ms	1
3783 (Curta CPU)	18402 ms	18423 ms	18627 ms	200 ms	225 ms	21 ms	25 ms	1
3804 (Curta CPU)	28040 ms	28213 ms	28416 ms	200 ms	376 ms	173 ms	176 ms	1

Turnaround Médio: 5071.12 ms
Tempo Resp. Médio: 169.72 ms
Tempo Espera Médio: 3371.75 ms
Eficiência Média: 33.51%
Trocas de Contexto/Tarefa: 8.69
Utilização da CPU: 88.09%

Este ultimo teste consolida os resultados do cenário anterior, mas agora agora tornando-os um pouco mais próximos do que aconteceria no mundo real. É possível evidenciar um comportamento preditivo e adaptativo do MLFQ sob uma carga de trabalho mais heterogênica.

Observamos que as tarefas de longa duração em “background”(Grupo 1) acumularam um número elevado de trocas de contexto (93/95) e tempo de espera elevado (35s), uma vez que cediam sistematicamente o processador para dar passagem para tarefas interativas e de curta duração que recém chegavam à readyqueue. As tarefas interativas com I/O (Grupo 3) e as tarefas curtas CPU-bound (Grupo 4) mantiveram tempos de resposta extremamente reduzidos (média de 169.72ms) e trocas de contexto mínimas (1 a 3), demonstrando a capacidade da MLFQ de lidar com comportamento interativo sem denegrir o progresso dos processos de fundo, garantindo uma taxa de utilização da CPU de 88.09%.

5. Conclusão

Em resumo, os resultados obtidos demonstram de forma clara o desempenho da Multi-Level Feedback Queue em cenários de execução reais e variados, ao equilibrar a otimização da latência de resposta para tarefas interativas com a distribuição justa de throughput para processos computacionalmente intensivos.

Os escalonadores clássicos como FIFO, SJF, SRTF e Round Robin servem primordialmente como fundamentos teóricos e mais acadêmicos (todos exigem um conhecimento prévio irrealista do tempo de execução das tarefas e falham na gestão de cargas concorrentes mistas). Já o MLFQ aprende e adapta-se dinamicamente ao comportamento de cada processo. E esta abordagem aproxima-se mais dos escalonadores modernos do mundo real, como o Completely Fair Scheduler (CFS) do Linux.

Embora o CFS utilize uma estrutura de árvore rubro-negra baseada em virtual runtime, conceito que não foi abordado diretamente neste trabalho, a sua dinâmica central une a adaptabilidade da MLFQ, a justiça distributiva do Round Robin e a priorização do menor tempo remanescente do SRTF. Isto demonstra que o estudo teórico destes algoritmos se torna indispensável para a compreensão sólida de arquiteturas de sistemas operacionais modernos, mesmo que as suas implementações puras raramente sejam aplicadas de forma isolada em cenários do mundo real.